

# Reviewer Pack — Minimal Rank of Tropical Curves Bounds Monomial Ideal Comple...

Entry 864710710971 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 23:20:23 UTC

## Minimal Rank of Tropical Curves Bounds Monomial Ideal Complexity for k-CLIQUE

Entry ID: 864710710971 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 23:20:23 UTC

### 1. Conjecture

**Field A** (mathematical branch): Tropical Geometry (Tropical Curves) **Field B** (complexity object): Complexity Theory: Monomial Ideal Complexity

#### Statement:

[‘For any monomial ideal  $I$  representing the  $k$ -CLIQUE problem, the minimal rank of its associated tropical curve is at least  $\Omega(n^2 \log n)$ .’, ‘This implies that for any  $k$ -CLIQUE instance with  $n$  variables, there exists a monomial ideal  $I$  such that the smallest possible rank of its tropical curve is not less than  $n$  squared times the logarithm of  $n$ .’, ‘If this conjecture holds, it would provide a lower bound on the complexity of solving  $k$ -CLIQUE in terms of the rank of the associated tropical curve.’]

#### Rationale (proposer’s reasoning):

[‘Tropical geometry has been shown to be useful for analyzing the complexity of certain computational problems by providing a geometric representation of computational structures.’, ‘Monomial ideals are closely related to the structure of Boolean functions and their complexity, making them an interesting object to study in this context.’, ‘By linking the rank of a tropical curve associated with a monomial ideal to the complexity of the  $k$ -CLIQUE problem, this conjecture may uncover new insights into the complexities of computational problems.’]

**Taxonomy category:** MONOTONE\_CLIQUE (status at proposal time: )

### 2. Pre-registration (Popper-style)

**Hash (SHA-256 prefix):** 09ae5d58a5684f46

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

#### Acceptance criterion:

The conjecture is supported if for all  $k$ -CLIQUE instances with  $n$  variables, the minimal rank of the tropical curve associated with the monomial ideal meets  $f(n) = \Omega(n^2 \log n)$  AND 95% CI of the mean minimal rank falls within  $[n^2 \log n \pm 1.96 * \sqrt{30/n^3}]$ . It

is falsified if there exists at least one instance where the minimal rank is less than  $n^2 \log n$ .

### 3. Barrier filter (F1)

**Final:** PASS

| Barrier        | Verdict | Confidence | LLM <sub>1</sub> | LLM <sub>2</sub> |
|----------------|---------|------------|------------------|------------------|
| RELATIVIZATION | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| NATURAL_PROOFS | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| ALGEBRIZATION  | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |
| KARP_LIPTON    | SAFE    | 0.50       | UNCERTAIN        | UNCERTAIN        |

### 4. Novelty audit

**Verdict:** ADJACENT\_OK against 3 hits across arXiv + Semantic Scholar + ECCC

**Search queries** (3): - "tropical geometry" AND "minimal rank" AND "monomial ideal complexity" - "k-clique problem" AND "tropical curve minimal rank" AND "ideal complexity" - "lower bound" ON solving k-CLIQUE AND "tropical curve rank" AND "logarithmic complexity"

**Top relevant hits considered:** - [<http://arxiv.org/abs/0906.0693v3>] An improved lower bound on the counterfeit coins problem - [<http://arxiv.org/abs/1007.1875v2>] Lower Bounds for Quantum Oblivious Transfer - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line

### 5. Empirical test harness

**Multi-seed protocol:** 5 seeds (11, 23, 37, 53, 71)

**Sandbox:** isolated subprocess, stdlib-only,  $\leq 90$ s wall time per cycle **Execution:** rc=1, elapsed=0.1s

#### 5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        m, n = len(A), len(A[0])
        rank = 0
        for j in range(n):
            i_max = rank
            for i in range(rank, m):
                if abs(A[i][j]) > abs(A[i_max][j]):
                    i_max = i
            if A[i_max][j] == 0:
                continue
            A[rank], A[i_max] = A[i_max], A[rank]
            for i in range(m):
                if i != rank:
                    factor = -A[i][j] / A[rank][j]
                    for k in range(n):
```

```

        A[i][k] += factor * A[rank][k]
    rank += 1
    return rank

def monomial_ideal_rank(n, k):
    # Generate a random k-CLIQUE instance
    vertices = list(range(n))
    edges = set()
    for _ in range(k):
        u = random.choice(vertices)
        v = random.choice(vertices)
        if u != v and (u, v) not in edges and (v, u) not in edges:
            edges.add((u, v))

    # Construct the associated monomial ideal
    I = []
    for edge in edges:
        i, j = edge
        monomial = [0] * n
        monomial[i] = 1
        monomial[j] = 1
        I.append(monomial)

    # Compute the tropical curve rank
    A = [[max(I[i][j], I[j][i]) for j in range(n)] for i in range(n)]
    return gaussian_elimination(A)

n_min, n_max = 5, 40
instances_tested = 0
total_rank = 0

for n in range(n_min, n_max + 1):
    rank = monomial_ideal_rank(n, k)
    if rank is None:
        return {
            "metric_name": "tropical_curve_rank",
            "metric_value": None,
            "instances_tested": instances_tested,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }
    total_rank += rank
    instances_tested += 1

mean_rank = Fraction(total_rank, instances_tested)
f_n = math.log(n_max) * n_max**2
if mean_rank < f_n:
    return {
        "metric_name": "tropical_curve_rank",
        "metric_value": float(mean_rank),
        "instances_tested": instances_tested,
        "conjecture_holds": False,
        "counterexample": f"mean rank {mean_rank} < {f_n}"
    }
else:
    return {

```

```

        "metric_name": "tropical_curve_rank",
        "metric_value": float(mean_rank),
        "instances_tested": instances_tested,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_rank = sum(result["metric_value"] for result in results) / len(results)
        std_dev = math.sqrt(sum((result["metric_value"] - mean_rank)**2 for result in results) / len(results))
        support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_rank} std={std_dev} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"mean rank < {math.log(n_max) * n_max**2}\" first failing seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

## 6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

## 7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce83caa5.py", line 105, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ce83caa5.py", line 68, in run_trial
    rank = monomial_ideal_rank(n, k)
                                   ^
NameError: name 'k' is not defined

```

## 8. Critic adversarial review

**Critic verdict:** CHALLENGE

**Critic reasoning:**

skipped: test crashed before producing data

## 9. Final verdict & safety rail

**Verdict:** INCONCLUSIVE

## Reasoning:

The test code crashed before producing data, which prevents us from verifying whether the conjecture’s support conditions are met. | next: Debug the test code to ensure it runs successfully and produces the required output for further analysis.

## 11. Audit log (LLM calls)

**Total LLM calls:** 12

| #  | Phase           | Provider      | Model            | Tokens in | out | Latency (ms) |
|----|-----------------|---------------|------------------|-----------|-----|--------------|
| 1  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 13448        |
| 2  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 11018        |
| 3  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 10562        |
| 4  | propose         | ollama_remote | glm4:latest      | 0         | 0   | 10782        |
| 5  | preregistration | ollama_remote | glm4:latest      | 0         | 0   | 6169         |
| 6  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 4970         |
| 7  | novelty         | ollama_remote | glm4:latest      | 0         | 0   | 7863         |
| 8  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 15961        |
| 9  | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10271        |
| 10 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 10482        |
| 11 | test_gen        | ollama_remote | qwen2.5-coder:7b | 0         | 0   | 11781        |
| 12 | judge           | ollama_remote | glm4:latest      | 0         | 0   | 9558         |

**Totals:** 0 input tokens, 0 output tokens, ~\$0.0000 cost, 122865 ms total latency. Provider mix: {'ollama\_remote': 12}

(full prompt+response transcripts available in `research/audit/864710710971.jsonl`)

## 12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/864710710971.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

**Sandbox file:** `research/pvsnp_sandbox/test_*.py` (per-cycle)

**Replay tarball:** `research/replay/864710710971.tar.gz` (if generated)